Abschlussprüfung Winter 2025

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit

Dashboard Entwicklung

Entwicklung eines webbasierten Health-Dashboards zur Projektüberwachung

Abgabetermin**:** 10.12.2025

Prüfungsbewerber:
Madian Aboalrish
Nordmeer Straße 47
21129 Hamburg

Prüflingsnummer:13154331

Umschulungsbetrieb;
BBQ Hamburg
Wendenstraße 23-29 | 20097 Hamburg

Ausbildungsbetrieb:
Christoph Backhaus IT / NADOOIT
Am Markt 1. 47229 Duisburg

Mobil: +49 176 565 44 075
E-Mail: christoph.backhaus@nadooit.de

Inhaltsverzeichnis

Abkürzungsverzeichnis 1
Abbildungsverzeichnis 2
1 Inhalt 1
1.1 Einleitung 1
1.2 Projektbeschreibung 1
1.3 Projektziel 1
1.4 Projektbegründung 2
1.5 Projektschnittstellen 2
1.6 Projektabgrenzung 2
2 Projektplanung 3
2.1 Projektphasen 3
2.2 Ressourcenplanung 3
2.3 Entwicklungsprozess 4
2.4 Risikoanalyse 4
3 Analysephase 4
3.1 Ist-Analyse 4
3.2 Wirtschaftlichkeitsanalyse 4
1. Make or Buy-Entscheidung 4
2. Projektkosten 5
3. Amortisationsdauer 5
3.3 Nutzwertanalyse 5
3.4 Anforderungsanalyse 5
3.5 Anwendungsfälle 6
3.6 Lastenheft 6
4 Entwurfsphase 6
4.1 Zielplattform (final version) 6
4.2 Architekturdesign 7
4.3 Entwurf der Benutzeroberfläche 7
4.4 Datenmodell 7
4.5 Geschäftslogik 8
4.6 Maßnahmen zur Qualitätssicherung 8
4.7 Deployment 9
4.8 Pflichtenheft 9
5 Implementierungsphase 9

5.1 Implementierung der Datenstrukturen 9
5.2 Implementierung der Geschäftslogik 10
5.3 Implementierung der Schnittstellen 10
5.4 Testen der Anwendung 10
5.5 Eingesetzte Technologien (Kurzfassung) 10
5.6 Qualitätssicherung (Kurzfassung) 11
6 Abnahmephase 11
6.1 Code-Review 11
6.2 Deployment und Einführung 11
6.3 Übergabe 12
7 Dokumentation 12
7.1 Projektdokumentation 12
7.2 Benutzerdokumentation 12
8 Fazit 12
8.1 Soll-/Ist-Vergleich 12
8.2 Lessons Learned 12
8.3 Ausblick 12
Literaturverzeichnis 1
Lastenheft 3
Pflichtenheft (Auszug) 4
1.A Sequenzdiagramm 5
2.A Aktivitaetsdiagramm 6
3.A Use-Case-Diagramm 7
4.A Datenmodell 7
5.A Arquitekturdesign 8
6.A Service-Klasse 8
7.A REST-Controller 8
8.A Repository-Schnittstelle 9
9.A Entity-Definition 9
10.A Projektstruktur 10
11.A Benutzeroberfläche 11

Abkürzungsverzeichnis

API ..... *Application Programming Interface*
CI/CD ..... *Continuous Integration / Continuous Delivery*
CSS ..... *Cascading Style Sheets*
DTO ..... *Data Transfer Object*
Git ..... *Versionsverwaltungssystem*
GitHub ..... *Plattform für Git-Repositorys*
HTML ..... *HyperText Markup Language*
HTTP ..... *Hypertext Transfer Protocol*
https ..... *Hypertext Transfer Protocol Secure*
JAR ..... *Java Archive*
JSON ..... *JavaScript Object Notation*
KPI ..... *Key Performance Indicator*
REST ..... *Representational State Transfer*
SSO ..... *Single Sign-On*
UI ..... *User Interface*

Abbildungsverzeichnis

2A Aktivitaetsdiagramm 6
3A Use-Case-Diagramm 7
4A Datenmodell 7
5A Arquitekturdesign 8
6A Service-Klasse 8
7A REST-Controller 8
8A Repository-Schnittstelle 9
9A Entity-Definition 9
10A Projektstruktur 10
11A Benutzeroberfläche 10

Tabellenverzeichnis

Tabelle 1: Grobe Zeitplanung (80 Stunden) 3
Tabelle 2: Risikoanalyse 4
Tabelle 3: Projektkosten (Kalkulation) 5
Tabelle 4: Zeiteinsparung pro Jahr 6
Tabelle 5: Nutzwertanalyse (Skala 1–10; je höher, desto besser) 6
Tabelle 6: Soll-/Ist-Vergleich 13
Tablle 7:Detaillierte Zeitplanung 2

1 Inhalt

1.1 Einleitung

In der folgenden Projektdokumentation wird das Abschlussprojekt des Autors im Rahmen der Abschlussprüfung zum Fachinformatiker für Anwendungsentwicklung (IHK) vorgestellt. Das Projekt wurde im Ausbildungsbetrieb NADOOIT / Christoph Backhaus IT durchgeführt.

Ziel des Projekts ist die Entwicklung eines webbasierten Health-Dashboards als Bestandteil des ProjectHub-Prototyps, das Projektdaten über interne REST-Schnittstellen automatisiert abruft, aufbereitet und übersichtlich visualisiert. Eine eigene persistente Datenbank ist im Projektumfang nicht vorgesehen.

Die Dokumentation beschreibt den Ablauf von der Planung und Analyse über Entwurf und Implementierung bis zur Abnahme

1.2 Projektbeschreibung

Im Rahmen des Projekts wird ein webbasiertes Health-Dashboard entwickelt, das der Projektleitung eine zentrale Übersicht über den aktuellen Zustand laufender Softwareprojekte bereitstellt. Hintergrund ist, dass im Unternehmen NADOOIT / Christoph Backhaus IT mehrere Projekte parallel betreut werden und relevante Kennzahlen (z. B. Fortschritt, Risiken, Aufwände und Tickets) derzeit verteilt in verschiedenen Tools vorliegen. Dadurch entsteht zusätzlicher Abstimmungsaufwand und ein konsistenter Gesamtüberblick ist nur eingeschränkt möglich.

Im Rahmen des Projekts wird ein webbasiertes Health-Dashboard entwickelt, das der Projektleitung eine zentrale Übersicht über den aktuellen Zustand laufender Softwareprojekte bereitstellt. Hintergrund ist, dass im Unternehmen NADOOIT / Christoph Backhaus IT mehrere Projekte parallel betreut werden und relevante Kennzahlen (z. B. Fortschritt, Risiken, Aufwände und Tickets) derzeit verteilt in verschiedenen Tools vorliegen. Dadurch entsteht zusätzlicher Abstimmungsaufwand und ein konsistenter Gesamtüberblick ist nur eingeschränkt möglich.

Das Dashboard soll die benötigten Informationen aus dem bestehenden Projektmanagement-System ProjectHub über eine REST-API automatisiert abrufen, aufbereiten und visuell darstellen. Projektverantwortliche sollen damit schnell erkennen können, ob Meilensteine eingehalten werden, welche Aufgaben kritisch sind und ob Risiken oder Terminabweichungen vorliegen. Die zentralen Anwendungsfälle sowie der technische Aufbau werden in den Abschnitten 3 Analysephase und 4 Entwurfsphase beschrieben.

Der Fachbereich erwartet insbesondere Transparenz, Zeitersparnis und eine frühzeitige Risikoerkennung. Aktuell werden Statusberichte manuell zusammengestellt, was zeitaufwändig und fehleranfällig ist. Durch die Automatisierung im Dashboard sollen Informationen konsistent bereitgestellt und die Entscheidungsfindung der Projektleitung unterstützt werden.

Ziel ist die Umsetzung einer prototypischen, produktionsnah ausführbaren Webanwendung im ProjectHub-Kontext, die relevante Projektdaten bündelt und grafisch aufbereitet.

1.3 Projektziel

Ziel dieses Projekts ist die Konzeption und Umsetzung eines webbasierten Health-Dashboards, das zentrale Projektdaten aus dem bestehenden System ProjectHub über eine REST-Schnittstelle automatisiert abruft, verarbeitet und übersichtlich visualisiert. Die Projektleitung soll dadurch alle relevanten Informationen an einer zentralen Stelle erhalten, um den manuellen Aufwand für Statusabfragen und Berichtserstellung zu reduzieren und Entscheidungen schneller sowie auf Basis aktueller Daten treffen zu können. Die fachlichen Funktionen werden in der Analysephase anhand der Anwendungsfälle und Anforderungen konkretisiert (siehe Abschnitt Analysephase). Die technische Realisierung wird im Entwurf über Systemarchitektur und Datenfluss beschrieben (siehe Abschnitt Entwurfsphase Entwurfsphase) und anschließend in der Implementierungsphase umgesetzt (siehe Abschnitt Implementierungsphase).

1.4 Projektbegründung

Der aktuelle Prozess zur Überwachung von Projektkennzahlen erfolgt überwiegend manuell. Relevante Informationen müssen aus dem System ProjectHub exportiert, anschließend zusammengeführt und in Excel aufbereiten werden. Dieser Ablauf ist zeitaufwändig und fehleranfällig, da bei manueller Übertragung und Zusammenführung der Daten regelmäßig Unstimmigkeiten entstehen können.

Durch die fehlende Automatisierung werden Abweichungen (z. B. Terminverschiebungen, kritische Tickets oder steigende Aufwände) häufig erst verspätet erkannt. Zudem existiert derzeit keine einheitliche, zentrale Sicht auf den aktuellen Projektstatus, was die Transparenz für die Projektleitung einschränkt und Entscheidungen verzögert.

Mit dem geplanten webbasierten Dashboard wird der Prozess automatisiert: Die benötigten Projektdaten werden über die REST-Schnittstellen abgerufen, aufbereitet und in einer zentralen Oberfläche dargestellt. Dadurch werden manuelle Tätigkeiten reduziert, Übertragungsfehler minimiert und eine schnelle, zuverlässige Übersicht über die wesentlichen Kennzahlen ermöglicht (siehe Abschnitt 3Analysephase).

1.5 Projektschnittstellen

Für die Umsetzung des Health-Dashboards werden mehrere Schnittstellen angebunden, um die relevanten Projektdaten automatisiert bereitzustellen. Die Anwendung bezieht die benötigten Informationen (z. B. Aufgaben, Fortschritt, Deadlines und Ressourcen) aus dem bestehenden System ProjectHub über eine REST-API, die strukturierten Daten im JSON-Format liefert. Die konkreten Aufruf- und Verarbeitungsabläufe werden im Entwurf sowie in der Implementierung beschrieben (siehe Abschnitt 4 Entwurfsphase und Abschnitt 5 Implementierungsphase).

Ergänzend können projektrelevante Kennzahlen wie Zeitaufwände, Statusmeldungen oder Risiken aus internen Quellen übernommen werden, sofern diese über definierte Endpunkte bereitgestellt werden. Die fachlichen Anforderungen an diese Daten sowie deren Priorisierung werden in der Analysephase festgelegt (siehe Abschnitt 3.4 Anforderungsanalyse und Abschnitt 3.6 Lastenheft).

Eine weitere Schnittstelle besteht zur unternehmensinternen Authentifizierung (z. B. Login/Token-Verfahren), um sicherzustellen, dass nur berechtigte Mitarbeitende Zugriff auf die Anwendung erhalten (siehe Abschnitt 3.4 Anforderungsanalyse).

Die Anforderungen an Kennzahlen und Benutzeroberfläche werden im Projektverlauf in Abstimmung mit Projektleitung und Stakeholdern konkretisiert. Die relevanten Anwendungsfälle sind in der Analysephase im Use-Case-Diagramm zusammengefasst (siehe Abschnitt Anwendungsfälle).

1.6 Projektabgrenzung

Der Projektumfang ist bewusst klar abgegrenzt. Bestandteil des Projekts ist die Entwicklung eines webbasierten Health-Dashboards inklusive der Anbindung an die vorhandenen ProjectHub-REST-Schnittstellen sowie der Visualisierung der daraus abgerufenen Projektdaten siehe Abschnitt.

Nicht Bestandteil dieses Projekts sind:
die Entwicklung einer eigenen Datenhaltung (z. B. persistente Datenbank oder Historisierung),
fortgeschrittene Analyseverfahren (z. B. Prognosen/Trendmodelle),
eine mobile App (Android/iOS),
automatisierte Benachrichtigungen (z. B. E-Mail/Push) sowie
Änderungen oder Erweiterungen am Quellsystem ProjectHub.

Diese Punkte können bei Bedarf als optionale Erweiterungen in einem späteren Projekt umgesetzt werden Abschnitt Fazit Ausblick.

2 Projektplanung

2.1 Projektphasen

Für die Umsetzung des Projekts standen insgesamt 80 Stunden zur Verfügung, entsprechend den Vorgaben der IHK. Vor Beginn der Entwicklung wurde das Gesamtvorhaben in mehrere Projektphasen gegliedert, die den vollständigen Ablauf der Softwareentwicklung abdecken.

Eine grobe Zeitplanung mit den Hauptphasen ist in Tabelle 1dargestellt. Die detaillierte Aufschlüsselung der einzelnen Arbeitsschritte befindet sich im Anhang (Detaillierte Zeitplanung).

Tabelle 1: Grobe Zeitplanung (80 Stunden)

Tabelle 1

Projektphase	**Geplante Zeit**
Anforderungsanalyse	10 h
Design & Planung	12 h
Backend-Entwicklung	14 h
Frontend-Integration	16 h
Integration & Testing	12 h
Review & Abnahme	6 h
Dokumentation	10 h
Gesamt	**80 h**

Tablle 1:Grobe Zeitplanung

2.2 Ressourcenplanung

Die für das Projekt benötigten Hard- und Softwareressourcen sind im Anhang A Aktivitaetsdiagrammaufgeführt. Es werden ausschließlich im Unternehmen bereits etablierte sowie lizenzfreie Komponenten eingesetzt, sodass keine zusätzlichen Lizenzkosten entstehen.

Das Backend wird mit Spring Boot umgesetzt und ruft die erforderlichen Projektdaten zur Laufzeit über interne REST-Schnittstellen des Systems ProjectHub ab. Eine eigene persistente Datenbank ist im Rahmen des Projekts nicht vorgesehen, da die Datenhaltung im Quellsystem erfolgt und das Dashboard primär der Aufbereitung und Visualisierung dient. Das Frontend wird mit HTML, CSS und JavaScript realisiert und übernimmt die Darstellung der aufbereiteten Informationen.

Für die Versionsverwaltung wird GitHub eingesetzt, um Änderungen nachvollziehbar zu dokumentieren und den Entwicklungsstand reproduzierbar zu halten. Die Durchführung erfolgt vollständig durch den Projektbearbeiter; zusätzliche personelle Ressourcen sind nicht eingeplant.

2.3 Entwicklungsprozess

Vor Beginn der Umsetzung wurde ein geeigneter Entwicklungsprozess festgelegt. Aufgrund der überwiegend klaren und im Projektantrag definierten Zielsetzung erfolgt die Umsetzung phasenorientiert nach dem Wasserfallmodell (Analyse → Entwurf → Implementierung → Test/Abnahme). Dadurch sind Planung, Dokumentation und Nachvollziehbarkeit für das Prüfungsprojekt sichergestellt .

Um insbesondere die Benutzeroberfläche praxisnah an die Anforderungen des Fachbereichs anzupassen, werden in der Entwurfs- und Umsetzungsphase kurze Feedback-Schleifen eingeplant. Rückmeldungen zur Darstellung der KPI und Diagramme werden gezielt aufgenommen und innerhalb der jeweiligen Phase umgesetzt, ohne die grundlegende A Projektstruktur zu verlassen siehe Abschnitt 4 und Abschnitt 5 .

Durch dieses Vorgehen wird ein strukturiertes, prüfungsgerechtes Arbeiten ermöglicht und gleichzeitig sichergestellt, dass die Ergebnisse die Erwartungen der Anwender erfüllen.

2.4 Risikoanalyse

Zur Vollständigkeit der Projektplanung wurde eine Risikoanalyse durchgeführt. Dabei wurden mögliche Risiken, deren Auswirkungen sowie geeignete Gegenmaßnahmen betrachtet, um eine stabile Durchführung des Projekts sicherzustellen (siehe Abschnitt ProjektphasenProjektphasen und Zeitplanung).

Tabelle 2: Risikoanalyse

Risiko	Auswirkung	Gegenmaßnahme
API nicht erreichbar	Dashboard kann keine Daten laden	Fehlerbehandlung im Backend, verständliche Fehlermeldung im Frontend, ggf. Retry-Mechanismus
Änderungen an der API	Datenverarbeitung/Anzeige fehlerhaft	Klare Schnittstellendefinition (DTOs), Validierung/Tests der API-Antworten
Zeitverzug im Frontend	Verzögerung der Integration	Priorisierung der Kernfunktionen (MVP), frühe Integration mit Testdaten
Hohe Datenmenge	Langsame Ladezeiten	Filter/Pagination, effiziente Abfragen und schlanke Datenobjekte

Tablle 2:kurze Risikoanalyse

3 Analysephase

3.1 Ist-Analyse

Aktuell werden projektbezogene Informationen wie Fortschritt, Aufgabenstatus, Risiken und Deadlines in mehreren Systemen und Kommunikationswegen gepflegt. Ein großer Teil der Daten befindet sich im internen System ProjectHub; ergänzende Informationen werden jedoch häufig manuell in Excel-Tabellen nachgeführt oder per E-Mail zwischen den Projektbeteiligten ausgetauscht. Dadurch entsteht eine verteilte Datenbasis ohne durchgängige, einheitliche Sicht auf den Projektstatus.

Diese Situation führt zu einem erhöhten manuellen Aufwand, insbesondere bei der Erstellung von Statusberichten: Kennzahlen müssen zusammengetragen, abgeglichen und plausibilisiert werden. Dadurch kommt es regelmäßig zu Verzögerungen, da Informationen nicht zentral verfügbar sind. Zudem besteht das Risiko, dass kritische Entwicklungen (z. B. Terminabweichungen oder überfällige Aufgaben) erst spät erkannt werden.

Automatisierte Prüfungen oder systematische Plausibilitätskontrollen sind im aktuellen Vorgehen nicht vorgesehen. Die Konsolidierung der Daten erfolgt überwiegend manuell, was die Fehleranfälligkeit erhöht und die Transparenz für die Projektleitung einschränkt. Die daraus abgeleiteten Anforderungen und Anwendungsfälle für die Ziel-Lösung werden in den Abschnitten 3.4 Anforderungsanalyse und 3.5 Anwendungsfälle beschrieben.

3.2 Wirtschaftlichkeitsanalyse

. Make or Buy-Entscheidung

Für die Umsetzung des Health-Dashboards wurde eine Make-or-Buy-Entscheidung getroffen. Die im Projekt verarbeiteten Informationen umfassen interne Projektkennzahlen und teilweise sensible Unternehmensdaten, die aus Datenschutz- und Sicherheitsgründen nicht an externe Dienstleister oder Cloud-Dienste übergeben werden sollen. Zudem muss die Lösung direkt mit dem bestehenden System ProjectHub über firmenspezifische REST-Schnittstellen kommunizieren siehe Abschnitt 1.5 Projektschnittstellen.

Standard-Dashboard- oder Reporting-Tools decken die erforderlichen Anforderungen nur eingeschränkt ab, da die benötigten KPIs, Visualisierungen sowie interne Prozesse individuell definiert sind. Eine Fremdlösung würde zusätzliche Integrationsaufwände (z. B. Schnittstellenanbindung,

Datenmapping, Berechtigungen) verursachen und zudem zu laufenden Lizenzkosten sowie einer stärkeren Abhängigkeit vom Anbieter führen.

Aus diesen Gründen wurde entschieden, das Health-Dashboard als Eigenentwicklung (Make) umzusetzen. Dadurch bleibt die Kontrolle über Datenhaltung im Quellsystem, Funktionalität und Weiterentwicklung im Unternehmen, und die Lösung kann gezielt an die fachlichen Anforderungen angepasst werden siehe Abschnitt 3 Analysephase und Abschnitt 4 Entwurfsphase.

. Projektkosten

Die im Projekt entstandenen Kosten basieren auf dem kalkulatorischen internen Stundensatz und dem im Projekt angefallenen Zeitaufwand. Die Kostenübersicht ist in Tabelle 3 dargestellt.

Tabelle 3: Projektkosten (Kalkulation)

Vorgang	Zeit	Kosten pro Stunde	Kosten
Entwicklung (80 h)	80 h	25,00 €	2.000,00 €
Review & Abnahme	6 h	40,00 €	240,00 €
Integration/Abstimmungen	3 h	40,00 €	120,00 €
Code-Review	2 h	40,00 €	80,00 €
Gesamtkosten			**2.440,00 €**

Table 3:Projektkosten

. Amortisationsdauer

Die Amortisationsdauer beschreibt den Zeitraum, nach dem sich die Investition in das Health-Dashboard durch die eingesparte Arbeitszeit wirtschaftlich ausgleicht. Die im Projekt entstandenen Projektkosten wurden in Tabelle 3 mit 2.440,00 € angesetzt.

Im aktuellen Prozess müssen projektbezogene Kennzahlen manuell aus verschiedenen Quellen zusammengestellt und aufbereitet werden. Dieser Vorgang dauert durchschnittlich 30 Minuten pro Arbeitstag. Durch die Einführung des Dashboards entfällt dieser manuelle Aufwand weitgehend, da die relevanten Daten automatisiert über die ProjectHub-Schnittstellen abgerufen und zentral visualisiert werden.

Die daraus resultierende jährliche Zeitersparnis ist in Tabelle 4 dargestellt.

Tabelle 4: Zeiteinsparung pro Jahr

Vorgang	Häufigkeit	Zeit alt	Zeit neu	Ersparnis pro Jahr
Manuelle Erstellung der Statusdaten	220 Arbeitstage/Jahr	30 Min/Tag	entfällt	6.600 Min (= 110 h)

Tablle 4:Zeiteinsparung

Gesamte Zeiteinsparung: 110 Stunden pro Jahr.
Auf Basis eines kalkulatorischen Stundensatzes von 40 €/h ergibt sich eine jährliche Einsparung von:
110 h × 40 €/h = 4.400,00 € pro Jahr
Damit amortisiert sich das Projekt nach:
2.440,00 € ÷ 4.400,00 €/Jahr ≈ 0,55 Jahre ≈ 6–7 Monate
Das Projekt amortisiert sich somit innerhalb des ersten Jahres und führt ab diesem Zeitpunkt zu einer jährlichen Entlastung.

3.3 Nutzwertanalyse

Neben den rein wirtschaftlichen Vorteilen bietet die Einführung des webbasierten Health-Dashboards insbesondere nicht-monetäre Mehrwerte, die im Projektumfeld eine zentrale Rolle spielen. Durch die automatisierte Datenübernahme und Aufbereitung werden bisher notwendige manuelle Arbeitsschritte reduziert, die fehleranfällig, zeitintensiv und teilweise von einzelnen Mitarbeitenden abhängig sind.

Die visuelle Aufbereitung der Projektkennzahlen erhöht die Transparenz über den aktuellen Projektzustand und unterstützt die Projektleitung dabei, Abweichungen (z. B. kritische Aufgaben oder Terminrisiken) schneller zu erkennen und geeignete Maßnahmen einzuleiten. Zusätzlich verbessert eine zentrale, einheitliche Darstellung die Nachvollziehbarkeit und Datenqualität, da Informationen konsistent aus dem Quellsystem bezogen werden.

Die nachfolgende Nutzwertanalyse stellt den Vergleich zwischen dem bisherigen manuellen Vorgehen und der automatisierten Dashboard-Lösung anhand definierter Kriterien dar (siehe Tabelle 5). Bewertet wird auf einer Skala von 1 (schlecht) bis 10 (sehr gut); die Gewichtung spiegelt die fachliche Relevanz wider. Der Nutzwert ergibt sich als gewichteter Durchschnitt.

Tabelle 5: Nutzwertanalyse (Skala 1–10; je höher, desto besser)

Eigenschaft	Gewichtung	Manuell	Automatisiert
Aktualität der Daten	8	2	8
Aufwand / Arbeitszeit	10	1	10
Fehleranfälligkeit	6	2	9
Transparenz der KPIs	4	3	9
Ausfallsicherheit / Stabilität	2	4	8
Summe	30	43	185

Table 5:Nutzwertanalyse

Nutzwert (gewichteter Durchschnitt):

Manuell: 43 / 30 = 1,43 (≈ 1,4)

Automatisiert: 185 / 30 = 6,17 (≈ 6,2)

Interpretation:

Die Analyse zeigt, dass die automatisierte Dashboard-Lösung in allen bewerteten Kriterien überlegen ist. Besonders stark wirken sich die höhere Aktualität der Daten, der reduzierte manuelle Aufwand sowie die geringere Fehleranfälligkeit aus. Damit bietet das Dashboard neben dem wirtschaftlichen Nutzen eine nachhaltige Qualitätssteigerung im Projektcontrolling.

3.4 Anforderungsanalyse

Auf Basis des Projektziels und der Ausgangssituation wurden die Anforderungen an das Health-Dashboard erhoben und in funktionale sowie nicht-funktionale Anforderungen unterteilt. Die Anforderungen wurden in Abstimmung mit der Fachabteilung und den beteiligten Stakeholdern definiert, um sicherzustellen, dass die Lösung den fachlichen Nutzen (Transparenz, Zeitersparnis und frühe Risikoerkennung) erfüllt (siehe Abschnitt 1.3 Projektziel und 3.1 Ist-Analyse).

Die Anforderungen wurden in einem Lastenheft dokumentiert und bilden die Grundlage für den anschließenden Entwurf und die Umsetzung (siehe Abschnitt 4 Entwurfsphase und Abschnitt 5 Implementierungsphase). Ein Auszug der wichtigsten Anforderungen ist im Anhang A Aktivitaetsdiagramm enthalten.

3.5 Anwendungsfälle

Die Interaktion zwischen den Anwendern und dem Health-Dashboard wurde anhand von Anwendungsfällen (Use Cases) beschrieben und in einem Use-Case-Diagramm modelliert. Dadurch werden die fachlichen Funktionen der Lösung klar abgegrenzt und nachvollziehbar dargestellt.

Zu den zentralen Anwendungsfällen gehören insbesondere:

das Abrufen der Projektdaten aus ProjectHub,
die Aufbereitung/Berechnung relevanter Kennzahlen sowie
die Visualisierung der Ergebnisse im Dashboard.

Das vollständige Use-Case-Diagramm ist im Anhang A Use-Case-Diagrammenthalten (siehe Use-Case-Diagramm im Anhang).

3.6 Lastenheft

Am Ende der Analysephase wurde gemeinsam mit dem Projektbetreuer ein Lastenheft erstellt, in dem die fachlichen und technischen Anforderungen an das zu entwickelnde Health-Dashboard dokumentiert wurden. Dieses Lastenheft bildet die verbindliche Grundlage für den anschließenden Entwurf und die Umsetzung.

Um eine termingerechte Umsetzung innerhalb des Projektzeitraums sicherzustellen, wurden die Anforderungen nach ihrer Relevanz für eine erste funktionsfähige Version priorisiert. Hierzu wurde die MoSCoW-Methode verwendet, bei der Anforderungen den Kategorien Must (muss), Should (soll), Could (kann) und Won’t (derzeit nicht / später wünschenswert) zugeordnet werden. Dadurch ist nachvollziehbar, welche Funktionen zwingend erforderlich sind und welche als optionale Erweiterungen in Betracht kommen .

Das vollständige Lastenheft befindet sich im Anhang A.6.

4 Entwurfsphase

4.1 Zielplattform

Das Health-Dashboard wird als eigenständige Webanwendung entwickelt und innerhalb der bestehenden IT-Infrastruktur von NADOOIT / Christoph Backhaus IT bereitgestellt. Der Betrieb erfolgt auf einem internen Server im Unternehmensnetzwerk. Dadurch ist kein externer Hosting-Dienst erforderlich und die Anwendung verbleibt vollständig innerhalb der internen Umgebung, was die Einhaltung von Sicherheits- und Datenschutzanforderungen unterstützt.

Eine eigene persistente Datenbank ist im Projektumfang nicht vorgesehen. Die Anwendung bezieht die benötigten Daten zur Laufzeit ausschließlich über interne ProjectHub-REST-APIs und verarbeitet diese für die Darstellung im Dashboard .

Für die Backend-Umsetzung wird Spring Boot eingesetzt, da die Technologie im Unternehmen etabliert ist und eine strukturierte Entwicklung REST-basierter Komponenten ermöglicht. Das Frontend wird mit HTML, CSS und JavaScript realisiert, um eine schlanke Integration sowie eine flexible Visualisierung der Kennzahlen zu gewährleisten. Die Versionsverwaltung erfolgt über GitHub.

Durch diese Zielplattform lässt sich das Dashboard wartungsarm betreiben und fügt sich in die bestehende Systemlandschaft ein.

4.2 Architekturdesign

Die Architektur des Health-Dashboards basiert auf einem klar strukturierten Schichtenmodell, das eine saubere Trennung von Zuständigkeiten ermöglicht. Das Backend wird mit Spring Boot umgesetzt und stellt die benötigten Funktionen über REST-Endpunkte bereit. Eine eigene persistente Datenbank ist im Projektumfang nicht vorgesehen, da die erforderlichen Daten zur Laufzeit über interne ProjectHub-APIs abgerufen werden .

Die Geschäftslogik ist in Service-Komponenten gekapselt. Dadurch werden Datenabruf, Aufbereitung und KPI-Berechnung unabhängig von der Darstellung umgesetzt und können getrennt

weiterentwickelt werden. Für den Datenaustausch werden strukturierte DTOs (z. B. Projekt-, Ticket- oder Zeitdaten) verwendet, um die Schnittstelle zwischen Backend und Frontend klar zu definieren.

Das Frontend greift über HTTP-Requests auf die REST-Schnittstellen zu und visualisiert die Kennzahlen mit HTML, CSS und JavaScript. Durch die Trennung von Backend (Datenbeschaffung/-aufbereitung) und Frontend (Darstellung) ist die Lösung gut erweiterbar: Neue KPIs oder zusätzliche Visualisierungen können ergänzt werden, ohne die grundlegende Architektur zu verändern.

Der Betrieb (Deployment) erfolgt auf einem internen Server innerhalb der bestehenden Infrastruktur. Da keine persistenten Daten gespeichert werden, ist ein schlanker Betrieb ohne zusätzliche Datenbankkomponenten möglich. Eine grafische Übersicht der Komponenten sowie der Ablauf des Datenabrufs sind im Anhang in den A Arquitekturdesign dokumentiert.

4.3 Entwurf der Benutzeroberfläche

die Gestaltung der Benutzeroberfläche wurden zunächst Mockups erstellt, um den grundlegenden Aufbau sowie die Darstellung der wichtigsten Kennzahlen frühzeitig mit dem Fachbereich Für abzustimmen. Die Mockups dienen als Grundlage für die spätere Implementierung und unterstützen die klare Abgrenzung zwischen fachlichen Anforderungen und konkreter Umsetzung (siehe Abschnitt Anforderungsanalyse).

Das Dashboard wird in die bestehende Systemlandschaft integriert und orientiert sich an einem einheitlichen, klar strukturierten Layout. Im Fokus stehen Übersichtlichkeit und eine intuitive Bedienung, sodass Projektstatus, Risiken und Deadlines ohne zusätzliche Navigation schnell erfassbar sind. Die fachlich relevanten Funktionen und Inhalte leiten sich aus den priorisierten Anforderungen sowie den Anwendungsfällen ab (siehe Abschnitt 3.5 Anwendungsfälle und 3.6)
.

Die Anwendung wird ausschließlich im internen Firmennetz auf Desktop-Arbeitsplätzen eingesetzt. Daher stehen Bedienbarkeit und Lesbarkeit auf großen Bildschirmen im Vordergrund; eine mobile Nutzung ist im aktuellen Projektumfang nicht vorgesehen . Die Oberfläche verwendet ein dunkles Farbschema, eine feste Navigationsleiste und klar getrennte Inhaltsbereiche. Zudem wird eine rollenbasierte Sicht umgesetzt, sodass bestimmte Bereiche nur bei entsprechender Berechtigung angezeigt werden.

4.4 Datenmodell

Das Health-Dashboard arbeitet im definierten Projektumfang ohne eigene persistente Datenbank. Alle benötigten Informationen werden zur Laufzeit über die interne Project Hub-REST-API bereitgestellt Abschnitt 1.5 Projektschnittstellen. Das Datenmodell besteht daher nicht aus gespeicherten Entitäten, sondern aus Datenstrukturen, die die JSON-Antworten der API abbilden und für die Darstellung aufbereitet werden.

Die API liefert pro Projekt unter anderem folgende Informationen:

- Projekt-ID und Projektname
- Status- und Fortschrittswerte
- Aufgaben/Tickets sowie offene Risiken
- Deadlines und verantwortliche Personen

Diese Daten werden im Backend in DTO-Klassen (Data Transfer Objects) überführt, die der Struktur der JSON-Antworten entsprechen. Auf dieser Basis erfolgt die fachliche Aufbereitung, z. B. die Berechnung aggregierter Fortschrittswerte, die Ermittlung kritischer Deadlines oder die Ableitung von Risikoindikatoren. Diese Logik ist im Backend in Service-Klassen gekapselt und damit klar von der Darstellung getrennt .

Da keine dauerhafte Speicherung vorgesehen ist und die Informationen bei jedem Abruf erneut aus ProjectHub geladen werden, ist kein Entity-Relationship-Modell erforderlich. Stattdessen basiert das Datenmodell auf temporären In-Memory-Strukturen, die aus den API-Antworten erzeugt werden. Die aufbereiteten Ergebnisse werden anschließend über die REST-Schnittstellen an das Frontend übertragen und dort visualisiert.

4.5 Geschäftslogik

Die Geschäftslogik des Health-Dashboards besteht darin, die über die ProjectHub-REST-API bereitgestellten Projektdaten zu validieren, aufzubereiten und zu den für die Darstellung benötigten Kennzahlen zu aggregieren. Dazu gehören insbesondere die Berechnung bzw. Zusammenführung von Fortschrittswerten, das Ermitteln kritischer Deadlines, das Ableiten von Risikoindikatoren sowie die Generierung weiterer Statusinformationen für die Projektleitung.

Da im Projekt keine eigene persistente Datenbank vorgesehen ist, erfolgt die gesamte fachliche Verarbeitung im Backend. Hierbei werden die eingehenden JSON-Daten zunächst in DTO-Strukturen überführt und anschließend in Service-Klassen verarbeitet (siehe Abschnitt 4.4 Datenmodell und Abschnitt 4.2 Architekturdesign). Erst die aufbereiteten, konsolidierten Ergebnisse werden über die REST-Schnittstellen an das Frontend übergeben und dort visualisiert.

4.6 Maßnahmen zur Qualitätssicherung

Während der Entwicklung des Health-Dashboards wurden gezielte Maßnahmen zur Qualitätssicherung umgesetzt, um fachliche Fehler frühzeitig zu erkennen und die Stabilität der Anwendung sicherzustellen. Da die Lösung auf klar definierten REST-API-Strukturen basiert, wurden Datenabrufe und Datenverarbeitung regelmäßig mit Testanfragen geprüft, um die korrekte Verarbeitung der API-Antworten sicherzustellen (siehe Abschnitt 1.5 und 4.4).

Ergänzend wurde der Quellcode in wesentlichen Arbeitsschritten durch den Ausbilder im Rahmen eines Code-Reviews kontrolliert. Dabei wurden sowohl technische Aspekte (z. B. Struktur, Fehlerbehandlung) als auch die fachliche Logik (z. B. KPI-Berechnung) bewertet. Die Versionsverwaltung über GitHub stellte sicher, dass Änderungen nachvollziehbar dokumentiert und bei Bedarf sauber zurückverfolgt werden konnten (siehe Abschnitt 2.2).

Das Frontend wurde fortlaufend im Browser getestet, insbesondere im Hinblick auf die korrekte Darstellung der Kennzahlen, die Aktualisierung der Werte sowie die Bedienbarkeit der Oberfläche. Die abschließende Überprüfung und Freigabe der Anwendung erfolgt im Rahmen der Abnahme und Einführung , siehe Abschnitt 6.2.

4.7 Deployment

Das Health-Dashboard wird als Spring-Boot-Anwendung auf einem internen Server innerhalb der Unternehmensinfrastruktur betrieben. Das Deployment erfolgt manuell, indem das erzeugte JAR-Artefakt auf dem Server ausgetauscht und die Anwendung anschließend neu gestartet wird.

Da die Anwendung ihre Daten zur Laufzeit über interne ProjectHub-REST-APIs bezieht und keine eigene persistente Datenhaltung vorgesehen ist, sind keine zusätzlichen Infrastrukturkomponenten (z. B. Datenbankserver) erforderlich . Nach dem Neustart stehen Updates unmittelbar zur Verfügung.

4.8 Pflichtenheft

Am Ende der Entwurfsphase wurde ein Pflichtenheft erstellt, das die technische Umsetzung der zuvor definierten Anforderungen konkret beschreibt. Es baut auf dem Lastenheft auf und überführt die dort festgelegten fachlichen Ziele in nachvollziehbare, umsetzbare Spezifikationen für das Health-Dashboard (Abschnitt 3.6).

Das Pflichtenheft dient während der Implementierung als Leitfaden, da es festlegt, wie die Anforderungen umgesetzt werden (z. B. über welche REST-Endpunkte Daten bezogen werden, wie die Aufbereitung der Kennzahlen erfolgt und welche Darstellungen im Frontend vorgesehen sind). Gleichzeitig ermöglicht es am Projektende eine eindeutige Prüfung, ob die Anforderungen vollständig umgesetzt wurden , siehe Abschnitt 6.2 sowie Abschnitt 6.3 Übergabe.

5 Implementierungsphase

Nach Abschluss der Entwurfsphase wurden die geplanten Strukturen des Health-Dashboards in die praktische Umsetzung überführt. In dieser Phase erfolgte die Implementierung des Backends mit den benötigten REST-Endpunkten sowie die Umsetzung der Frontend-Oberfläche zur Darstellung der

Kennzahlen. Grundlage hierfür waren das Lasten- und Pflichtenheft sowie der in der Entwurfsphase definierte Architektur- und Datenfluss (siehe Abschnitt 3.6 Lastenheft und Abschnitt 4 Entwurfsphase).

5.1 Implementierung der Datenstrukturen

Die für das Health-Dashboard benötigten Datenobjekte wurden als Java-Klassen modelliert und orientieren sich an den Strukturen der von ProjectHub gelieferten JSON-Daten. Zu den zentralen Datenstrukturen gehören beispielsweise ProjectInfo, Task, ProjectStatus und ProjectStep. Diese Klassen bilden die wichtigsten Informationen eines Projekts ab, wie Status- und Fortschrittswerte, anstehende Aufgaben, Deadlines sowie verantwortliche Teammitglieder (siehe Abschnitt 4.4 Datenmodell).

Die Objekte werden im Backend als DTOs verarbeitet: Spring Boot übernimmt die automatische Zuordnung der API-Antworten zu den Java-Klassen (Mapping/Deserialisierung). Anschließend werden die Daten in den Service-Klassen fachlich aufbereitet und zu den für das Dashboard benötigten Kennzahlen aggregiert (siehe Abschnitt 4.5 Geschäftslogik). Eine persistente Speicherung ist im Projekt nicht vorgesehen, da die Daten bei jedem Aufruf zur Laufzeit über die ProjectHub-API abgerufen werden.

Die aufbereiteten Ergebnisse werden anschließend über REST-Endpunkte an das Frontend übergeben und dort visualisiert.

5.2 Implementierung der Geschäftslogik

Die Geschäftslogik des Health-Dashboards ist in der Service-Schicht umgesetzt. Dort werden die zentralen Abläufe verarbeitet, insbesondere das Abrufen der Projektdaten aus ProjectHub, die fachliche Aufbereitung der Daten sowie die Ermittlung des aktuellen Projektstatus anhand definierter Kennzahlen (siehe Abschnitt 4.5 Geschäftslogik).

Die Services beziehen die Daten zur Laufzeit über die internen ProjectHub-REST-Schnittstellen und überführen die eingehenden JSON-Daten zunächst in DTO-Strukturen. Anschließend werden die Daten validiert, konsolidiert und für die Darstellung aggregiert. wie Berechnung von Fortschrittswerten, Ermittlung kritischer Deadlines und Ableitung von Risikoindikatoren). Eine persistente Speicherung ist nicht vorgesehen; die Verarbeitung erfolgt vollständig in Memory

Eingehende REST-Anfragen an das Backend werden über Controller entgegengenommen, geprüft und an die Service-Schicht weitergeleitet. Die aufbereiteten Ergebnisse werden anschließend in einer für das Frontend geeigneten Struktur zurückgegeben, sodass die Darstellung im Dashboard konsistent erfolgen kann.

5.3 Implementierung der Schnittstellen

Die Kommunikation zwischen Frontend und Backend erfolgt im Health-Dashboard über REST-Schnittstellen. Das Spring-Boot-Backend stellt dafür definierte Endpunkte bereit, die vom Frontend über die Fech-API aufgerufen werden. Die Daten werden im JSON-Format übertragen und anschließend im JavaScript verarbeitet, um die Inhalte dynamisch im Dashboard darzustellen.

Durch dieses Vorgehen entsteht eine klare Trennung der Verantwortlichkeiten: Das Backend übernimmt den Abruf und die Aufbereitung der Projektdaten , während das Frontend ausschließlich für Darstellung und Benutzerinteraktion zuständig ist. Der Ablauf eines typischen Datenabrufs (Frontend → Backend → Project Hub-API → Rückgabe) entspricht dem im Entwurf definierten Datenfluss (siehe Abschnitt 4.2 Architekturdesign).

5.4 Testen der Anwendung

Die Funktionsfähigkeit des Health-Dashboards wurde im Projektverlauf durch manuelle Tests überprüft. Dabei lag der Schwerpunkt auf der Prüfung der REST-Schnittstellen sowie der korrekten Verarbeitung und Darstellung der Projektdaten im Dashboard. Es wurde kontrolliert, ob Daten vollständig geladen, korrekt aufbereitet und in der Benutzeroberfläche erwartungsgemäß angezeigt werden.

Zusätzlich wurden typische Fehler- und Grenzfälle geprüft, um die Stabilität der Anwendung sicherzustellen. Dazu gehörten beispielsweise leere Ergebnislisten, fehlende Werte in API-Antworten sowie nicht erfolgreiche API-Aufrufe, bei denen eine verständliche Fehlermeldung im Frontend ausgegeben wird, Abschnitt 2.4.

Die abschließende Überprüfung und Freigabe der Anwendung erfolgt im Rahmen der Einführung und Abnahme. siehe Abschnitt 6.2. Die hier beschriebenen Tests dienten insbesondere der fortlaufenden Qualitätssicherung während der Implementierung

5.5 Eingesetzte Technologien

Das Health-Dashboard wurde im Backend mit Spring Boot umgesetzt. Spring Boot übernimmt den Abruf der Projektdaten aus ProjectHub, deren Aufbereitung sowie die Bereitstellung der internen REST-Schnittstellen für das Frontend.

Im Frontend werden HTML, CSS und JavaScript eingesetzt. Die Kommunikation mit dem Backend erfolgt über die Fetch-API; die Daten werden im JSON-Format übertragen und anschließend für die Darstellung im Dashboard verarbeitet.

Eine persistente Datenbank ist im Projektumfang nicht vorgesehen, da die Daten zur Laufzeit über die ProjectHub-APIs bezogen werden (siehe Abschnitt 4.1 Zielplattform sowie Projektabgrenzung in Kapitel 1).

5.6 Qualitätssicherung (Kurzfassung)

Zur Sicherstellung der Softwarequalität wurden während der Entwicklung regelmäßige Funktionsprüfungen durchgeführt. Dabei wurde insbesondere kontrolliert, ob die REST-Schnittstellen erwartungsgemäß reagieren und die abgerufenen Daten konsistent verarbeitet sowie korrekt im Dashboard dargestellt werden. Abschnitt 4.6 Maßnahmen zur Qualitätssicherung und 5.4 Testen der Anwendung.

Zusätzlich wurden typische Fehler- und Grenzfälle (z. B. leere Ergebnislisten oder fehlerhafte API-Antworten) überprüft, um die Stabilität der Anwendung sicherzustellen (siehe Abschnitt 2.4 Risikoanalyse). Perspektivisch können automatisierte Tests Unit-Tests der Service-Logik und API-Tests) ergänzt werden, um die Qualität langfristig weiter abzusichern (siehe Abschnitt 8 Ausblick).

6 Abnahmephase

In der Abnahmephase wurde die Anwendung final überprüft, fachlich abgestimmt und für die interne Nutzung freigegeben. Dazu gehörten Code-Review, Bereitstellung in einer produktionsnahen Umgebung sowie die Übergabe an den betreuenden Ansprechpartner.

6.1 Code-Review

Vor der finalen Vorstellung der Anwendung wurde ein Code-Review gemeinsam mit dem betreuenden Ausbilder durchgeführt. Dabei lag der Schwerpunkt auf Lesbarkeit, Struktur, Nachvollziehbarkeit sowie der korrekten Umsetzung der fachlichen Logik (z. B. Aufbereitung der Projektdaten und KPI-Berechnung).

Die im Review identifizierten Hinweise und Verbesserungsvorschläge (z. B. zur Code-Struktur, Fehlerbehandlung oder Benennung) wurden anschließend direkt in der Umsetzung berücksichtigt. Dadurch konnte sichergestellt werden, dass der Quellcode sauber aufgebaut ist und den internen Qualitätsanforderungen entspricht , Abschnitt 4.6.

6.2 Deployment

Nach Abschluss der Tests wurde das Health-Dashboard auf der internen Serverumgebung bereitgestellt Die Anwendung wurde in einer produktionsnahen Umgebung ausgeführt, um die Stabilität sowie das Zusammenspiel der angebundenen REST-Schnittstellen zu überprüfen (siehe Abschnitt 5.3 Implementierung der Schnittstellen).

Im Anschluss erfolgte eine kurze Einweisung der Anwender, in der die wichtigsten Funktionen des Dashboards vorgestellt wurden, insbesondere Projektübersicht, Fortschrittsanzeige sowie die Anzeige

kritischer Aufgaben und Deadlines. Nach erfolgreicher Überprüfung durch das Team wurde die Anwendung freigegeben.

6.3 Übergabe

Nach der erfolgreichen Abnahme wurde das Health-Dashboard an den betreuenden Ausbilder bzw. den verantwortlichen Ansprechpartner übergeben. Im Rahmen der Übergabe wurden die Projektstruktur, die wichtigsten Funktionen sowie der Ablauf der Datenbeschaffung und -aufbereitung erläutert (siehe Abschnitt 4.2 Architekturdesign und 5.2 Implementierung der Geschäftslogik).

Zusätzlich wurden alle relevanten Projektartefakte bereitgestellt, insbesondere Quellcode, Konfigurationsdateien sowie die Projektdokumentation. Mit der Übergabe wurde das Projekt formal abgeschlossen und für die interne Nutzung freigegeben.

7 Dokumentation

7.1 Projektdokumentation

Die Projektdokumentation fasst den Ablauf und die Ergebnisse des Projekts zur Entwicklung des Health-Dashboards strukturiert zusammen. Sie beschreibt die wesentlichen Phasen von der Anforderungsanalyse über Entwurf und Implementierung bis hin zu Test, Abnahme und Übergabe (siehe Kapitel 3 Analysephase, Kapitel 4 Entwurfsphase, Kapitel 5 Implementierungsphase sowie Kapitel 6 Abnahmephase).

Durch die nachvollziehbare Darstellung der Entscheidungen, Schnittstellen und Umsetzungsdetails entsteht eine vollständige Grundlage, um den Projektverlauf sowie den technischen Aufbau der Anwendung zu verstehen und die Zielerreichung zu beurteilen.

7.2 Benutzerdokumentation

Die Benutzerdokumentation richtet sich an die Anwender des Health-Dashboards und beschreibt die grundlegende Bedienung der Anwendung. Sie unterstützt die Nutzer dabei, sich schnell zurechtzufinden, und enthält kurze Anleitungen zur Navigation, zur Anzeige der Projektinformationen sowie zur Interpretation der dargestellten Kennzahlen.

Typische Abläufe (z. B. Projektübersicht aufrufen, Detailansicht prüfen, kritische Deadlines identifizieren) werden anhand von Screenshots erläutert, die im Anhang dokumentiert sind. Dadurch ist die Anwendung auch ohne zusätzliche Einweisung nachvollziehbar nutzbar.

8 Fazit

8.1 Soll-/Ist-Vergleich

Für das Projekt wurde zu Beginn ein geplanter Gesamtaufwand von 80 Stunden festgelegt. Während der Durchführung kam es zu kleineren zeitlichen Abweichungen, die jedoch den vorgesehenen Rahmen nicht überschritten. Der Vergleich basiert auf der detaillierten Zeitplanung (siehe Anhang A Sequenzdiagramm).

Tabelle 6: Soll-/Ist-Vergleich

Phase	Geplant	Tatsächlich	Differenz
Analysephase	5 h	6 h	+1 h
Entwurfsphase	13 h	13 h	0 h
Implementierungsphase	40 h	41 h	+1 h

Abnahme- und Einführungsphase	5 h	4 h	-1 h
Dokumentationsphase	17 h	16 h	-1 h
Gesamt	**80 h**	**80 h**	**0 h**

Table 6:Soll-/Ist-Vergleich

Die entstandenen Abweichungen erklären sich hauptsächlich durch zusätzlichen Aufwand bei der Implementierung, insbesondere bei der Verarbeitung und Aufbereitung der Projektdaten. Gleichzeitig konnte in anderen Phasen Zeit eingespart werden, sodass der Gesamtumfang von 80 Stunden exakt eingehalten wurde.

8.2 Lessons Learned

Im Rahmen des Projekts konnte ich meine Kenntnisse im Umgang mit REST-Schnittstellen und der Verarbeitung von JSON-Daten vertiefen. Insbesondere die strukturierte Datenaufbereitung im Backend und die klare Trennung von Geschäftslogik und Darstellung haben mein Verständnis für eine wartbare Schichtenarchitektur verbessert (Abschnitt 4.2 Architekturdesign und 5.2 Implementierung der Geschäftslogik).

Bei der Umsetzung der Benutzeroberfläche mit HTML, CSS und JavaScript wurde deutlich, wie wichtig eine klare Struktur und eine übersichtliche Darstellung für ein Dashboard sind. Rückmeldungen aus der Abstimmung führten zu gezielten Anpassungen an Kennzahlen und Fortschrittsanzeigen, wodurch die Bedienbarkeit verbessert werden konnte (siehe Abschnitt 4.3 Entwurf der Benutzeroberfläche).

Organisatorisch hat sich die Aufteilung in kleine, priorisierte Arbeitsschritte bewährt. Durch regelmäßige Funktionsprüfungen und frühe Integration der Schnittstellen konnten Fehler frühzeitig erkannt und der Zeitplan stabil eingehalten werden. Insgesamt konnte ich sowohl technische als auch methodische Fähigkeiten weiterentwickeln und wertvolle Erfahrungen für zukünftige Projekte sammeln.

8.3 Ausblick

Auch wenn die zentralen Anforderungen des Health-Dashboards umgesetzt wurden, sind zukünftige Erweiterungen sinnvoll und realistisch:

Erweiterung der Kennzahlen: Bei Bedarf können zusätzliche KPIs z. B. Team-Performance, Ticket-Durchlaufzeiten oder Kapazitätsauslastung ergänzt werden, ohne die bestehende Struktur grundlegend zu verändern.

Integration weiterer Datenquellen: Das Backend kann um weitere interne REST-APIs erweitert werden, um zusätzliche Projekt- oder Mitarbeiterdaten einzubinden.

Technische Weiterentwicklung des Betriebs: Eine spätere Einführung eines automatisierten Build- und Deployment-Prozesses CI/CD mit GitHub Actions würde Updates reproduzierbarer machen und den Betrieb vereinfachen.

Optimierung der Benutzeroberfläche: Zusätzliche Visualisierungen und interaktive Diagramme können die Auswertung weiter verbessern und die Bedienbarkeit erhöhen.

Zusammenfassend ist die Lösung modular aufgebaut und bietet damit eine stabile Grundlage, um das Dashboard künftig an neue Anforderungen anzupassen und kontinuierlich weiterzuentwickeln.

Literaturverzeichnis

[1] Spring Boot – Referenzdokumentation
https://docs.spring.io/spring-boot/docs/current/reference/html/

[2] Spring Framework – Web MVC (REST, Controller, HTTP)
https://docs.spring.io/spring-framework/reference/web/webmvc.html

[3] Spring Framework – REST Clients (RestClient/RestTemplate/WebClient)
https://docs.spring.io/spring-framework/reference/integration/rest-clients.html

[4] Jackson Databind – GitHub Repository (JSON Mapping/Deserialisierung)
https://github.com/FasterXML/jackson-databind

[5] MDN Web Docs – Fetch API
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[6] MDN Web Docs – HTTP-Statuscodes
https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

[7] RFC 9110 – HTTP Semantics
https://www.rfc-editor.org/rfc/rfc9110

[8] OWASP Cheat Sheet Series – REST Security Cheat Sheet
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

[9] H2 Database Engine – Dokumentation *(nur falls im Projekt verwendet; andernfalls entfernen)*
https://www.h2database.com/html/main.html

Tablle 7:Detaillierte Zeitplanung

Phase	Aufgabe	Dauer
Analysephase	4 h	
	1. Ist-Analyse durchführen	2 h
	2. Anforderungsanalyse (Use-Cases, Ziele)	2 h
Entwurfsphase	13 h	
	1. Schnittstellendesign Backend-API	2 h
	2. Recherche zu Dashboard-Technologien	3 h
	3. Entwurf Datenstrukturen (DTOs, Models)	2 h
	4. Architektur & Routingplanung	2 h
	5. Entwurf Benutzeroberfläche (Wireframes)	4 h
Implementierungsphase	42 h	
	1. Backend-API-Endpunkte erweitern (ProjectHub)	8 h
	2. Datenvalidierung & Geschäftslogik	6 h
	3. Fetch-Integration & JSON-Parsing	5 h
	4. Implementierung Dashboard-Views	10 h
	5. Fehler- und Statusmeldungen (Toast)	3 h
	6. Performance-Optimierungen	3 h
	7. SSO-Integration vorbereiten (Token Handling)	4 h
Qualitätsmanagement	7 h	
	1. Funktionstests	2 h
	2. UX-Überprüfung	2 h
	3. Fehlerbehebungen	3 h
Abnahme & Einführung	5 h	
	1. QS-Review & Feedbackrunde	2 h
	2. Final Pull Request & Merge	1 h
	3. Produktivnahme (interne Umgebung)	2 h
Dokumentation	9 h	
	1. Projektdokumentation	6 h
	2. Screenshots & Codebeispiele	2 h
	3. Diagramme & Architekturübersicht	1 h

Tablle 7:Detaillierte Zeitplanung

Lastenheft

Anforderungen an Datenabruf und Verarbeitung

1.1 Die Anwendung muss Projektdaten über die bereitgestellte Schnittstelle (ProjectHub-REST-API) abrufen.
1.2 Die abgerufenen Daten müssen serverseitig validiert, verarbeitet und in einem einheitlichen JSON-Format für das Frontend bereitgestellt werden.
1.3 Kennzahlen (z. B. Projektstatus, Fortschritt, kritische Deadlines und Risiken) müssen automatisiert aus den Projektdaten abgeleitet werden.
1.4 Die Verarbeitung muss performant erfolgen, sodass die Nutzung im Tagesgeschäft nicht beeinträchtigt wird.

Anforderungen an Darstellung und Bedienung

2.1 Das Dashboard muss eine übersichtliche Projektübersicht bereitstellen (z. B. Status, Fortschritt, Tasks/Tickets, Deadlines).
2.2 Kritische Zustände (z. B. Risiken, Verzögerungen, überfällige Deadlines) müssen visuell erkennbar sein.
2.3 Die Benutzeroberfläche muss verständlich und strukturiert gestaltet sein.
2.4 Die Aktualisierung der angezeigten Daten muss ohne vollständiges Neuladen der Seite möglich sein.

Nicht-funktionale Anforderungen und Randbedingungen

3.1 Die Anwendung muss im Firmennetz über einen gängigen Desktop-Browser erreichbar sein.
3.2 Die Datenübertragung soll verschlüsselt erfolgen (HTTPS), sofern dies durch die interne Infrastruktur bereitgestellt wird.
3.3 Der Zugriff auf geschützte Bereiche darf nur nach erfolgreicher Authentifizierung/Autorisierung erfolgen (z. B. Token-/SSO-Verfahren gemäß Unternehmensvorgaben).
3.4 Die Anwendung soll wartbar und erweiterbar sein (klare Trennung von Backend und Frontend).
3.5 Eine eigene persistente Datenbank ist nicht Bestandteil des Projekts; die Datenhaltung verbleibt im Quellsystem.

Pflichtenheft (Auszug)

Plattform

1.1 Die Anwendung wird als browserbasierte Webanwendung umgesetzt.
1.2 Das Frontend wird mit HTML, CSS und JavaScript realisiert.
1.3 Das Backend wird als Spring-Boot-Anwendung innerhalb der internen Infrastruktur betrieben.
1.4 Der Zugriff erfolgt über aktuelle Desktop-Webbrowser im Firmennetz.
1.5 Eine lokale Installation auf Endgeräten ist nicht erforderlich.

Backend und Schnittstellen

2.1 Das Backend stellt REST-Endpunkte zur Versorgung des Frontends bereit.
2.2 Die erforderlichen Projektdaten werden zur Laufzeit über die ProjectHub-REST-API abgerufen.
2.3 Eine eigene persistente Datenbank wird nicht betrieben; die Datenhaltung erfolgt im Quellsystem.
2.4 Die serverseitige Verarbeitung umfasst Validierung, Konsolidierung und Aggregation der Daten (inkl. KPI-Berechnung).
2.5 Das Backend liefert die aufbereiteten Ergebnisse als JSON an das Frontend zurück.

Oberfläche

3.1 Die Oberfläche stellt Projektübersicht und projektbezogene Detailinformationen übersichtlich dar.
3.2 Kritische Zustände werden optisch hervorgehoben (z. B. Statusindikatoren/Markierungen).
3.3 Daten können dynamisch aktualisiert werden (Fetch/AJAX), ohne vollständigen Seiten-Reload.

Geschäftslogik

4.1 Die fachliche Logik (KPI-Berechnungen, Risikoindikatoren, Plausibilitätsprüfungen) wird serverseitig in Service-Komponenten umgesetzt.
4.2 Das Frontend übernimmt Darstellung und Benutzerinteraktion (z. B. Navigation, Filter, Aktualisierung).
4.3 Fehler- und Statusmeldungen werden nutzerverständlich in der Oberfläche ausgegeben.

Sicherheit

5.1 Die Kommunikation erfolgt verschlüsselt (HTTPS), sofern die interne Infrastruktur dies bereitstellt.
5.2 Zugriff auf geschützte Bereiche erfolgt gemäß Unternehmensvorgabe (z. B. Token/SSO).
5.3 Sicherheitsrelevante Prüfungen (Autorisierung/Validierung) erfolgen im Backend.

Qualitätssicherung

6.1 Zentrale Funktionen werden während der Entwicklung manuell getestet (API-Aufrufe, Fehlerfälle, UI-Darstellung).
6.2 Ein Code-Review durch den Ausbilder dient der Kontrolle von Struktur, Verständlichkeit und fachlicher Logik.

1. A Sequenzdiagramm

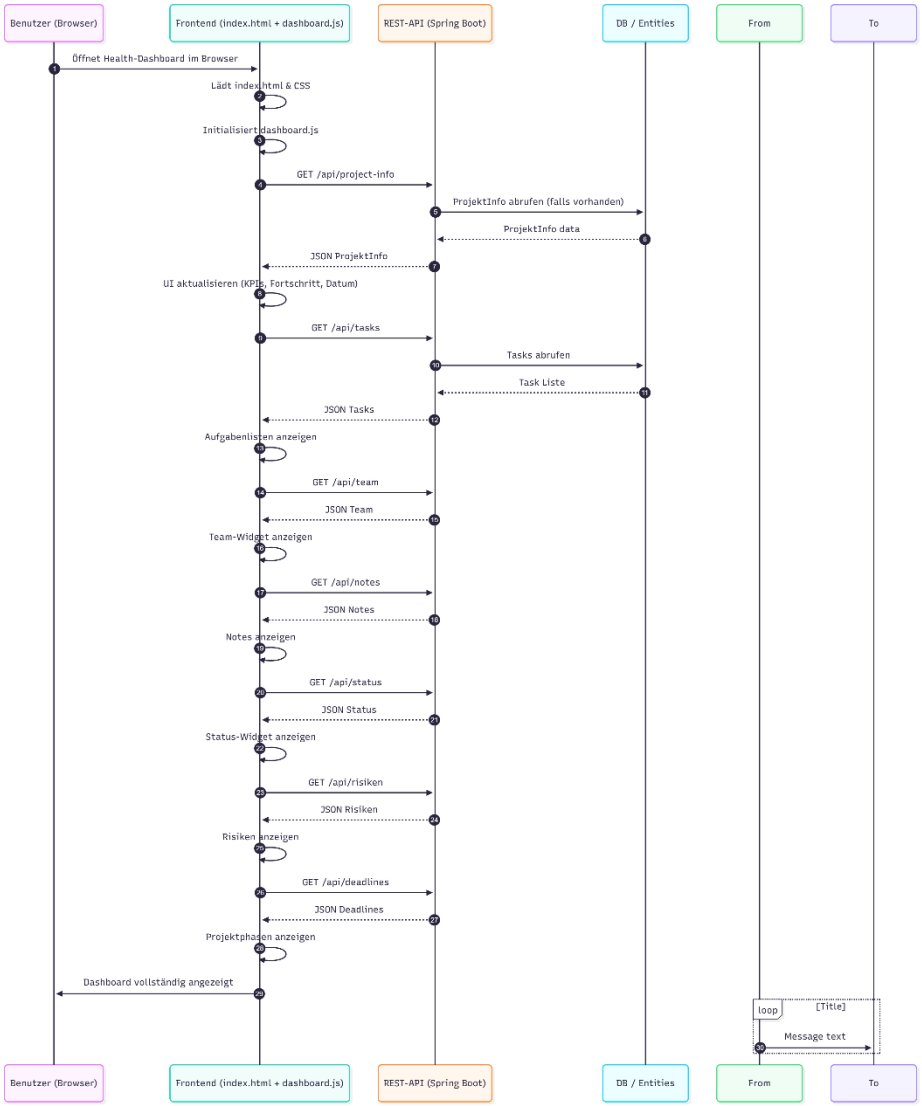

Abbildung 1:Sequenzdiagramm

2. A Aktivitaetsdiagramm

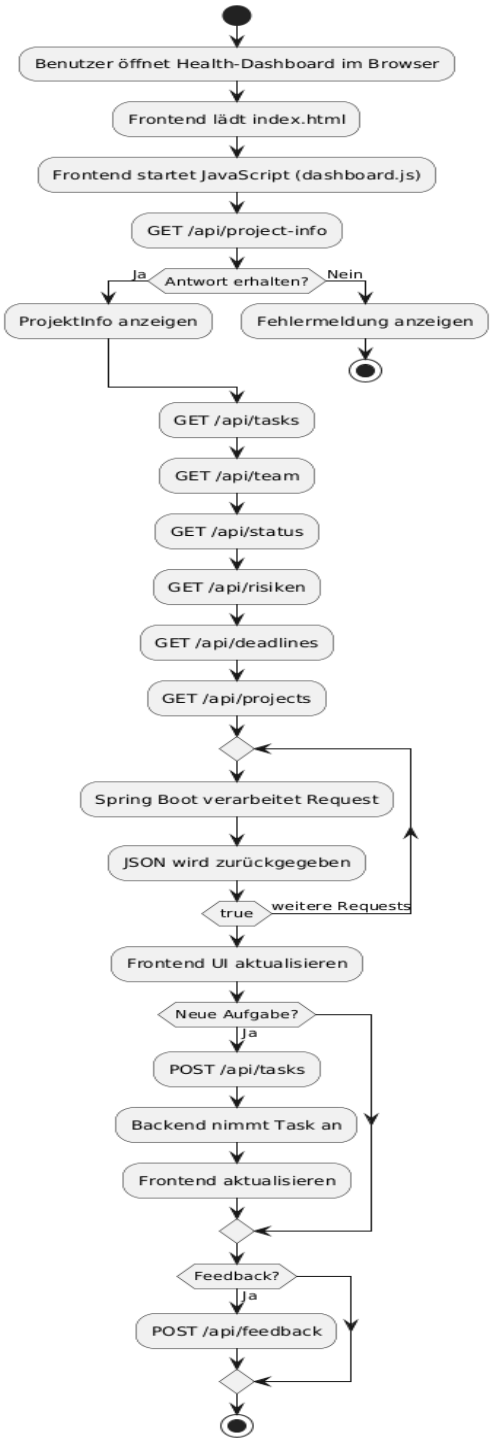

Abbildung 2:Aktivitaetsdiagramm

3. A Use-Case-Diagramm

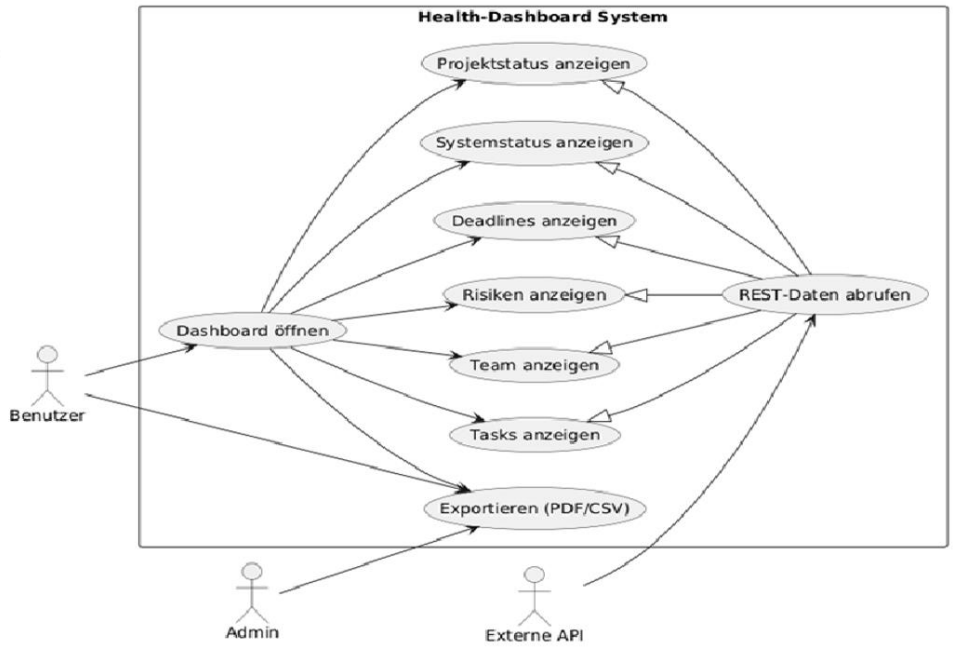

Abbildung 3:Use-Case-Diagramm

4. A Datenmodell

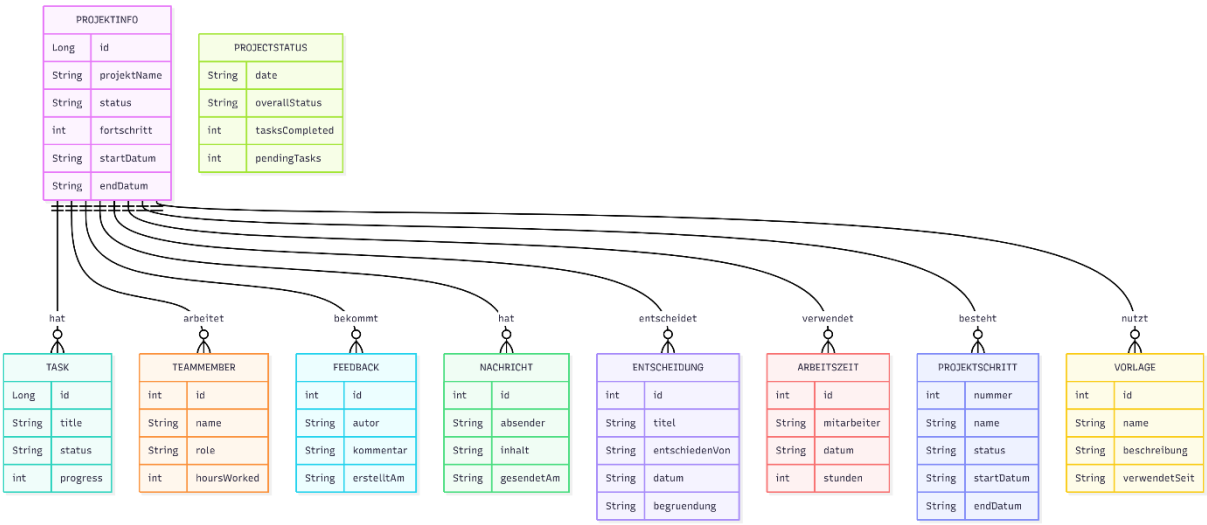

Figure 1:Datenmodell

5. A Arquitekturdesign

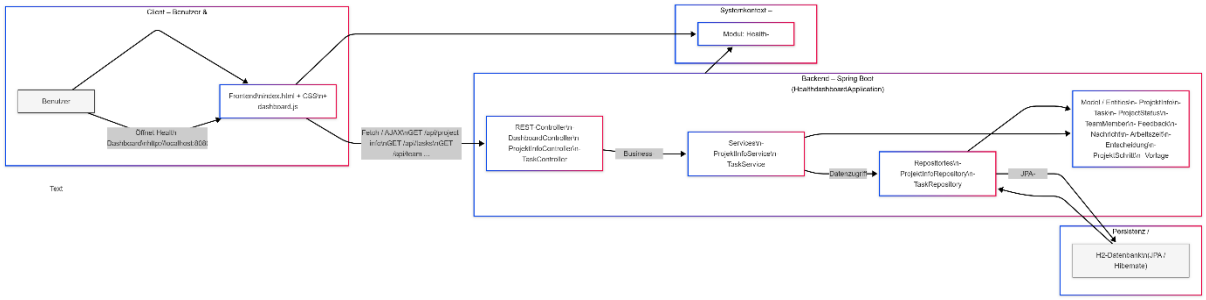

Figure 2:Arquitekturdesign

6. A Service-Klasse

```
@Service  3 usages
public class ProjektInfoService {

    private final ProjektInfoRepository projektInfoRepository;  6 usages

    public ProjektInfoService(ProjektInfoRepository projektInfoRepository) {  no usages
        this.projektInfoRepository = projektInfoRepository;
    }

    public List<ProjektInfo> findAll() {  2 usages
        // Gibt alle Projekte aus der Datenbank zurück.
        return projektInfoRepository.findAll();
    }
```

Figure 3:Service-Klasse

7. A REST-Controller

```
// PROJECT INFO

    @GetMapping("/api/project-info")  no usages
    public Map<String, Object> getProjectInfo() {
        Map<String, Object> data = new HashMap<>();
        data.put("projectName", "Health Dashboard");
        data.put("status", "OK");
        data.put("progress", 40);
        return data;
    }
```

Figure 4:REST-Controller

8. A Repository-Schnittstelle

```
1   package com.projecthub.healthdashboard.repository;
2
3   import com.projecthub.healthdashboard.model.ProjektInfo;
4   import org.springframework.data.jpa.repository.JpaRepository;
5   import org.springframework.stereotype.Repository;
6
7   @Repository  3 usages
8   public interface ProjektInfoRepository extends JpaRepository<ProjektInfo, Long> {
9   }
10
```

Figure 5:Repository-Schnittstelle (JPA)

9. A Entity-Definition

```
12  @Entity  17 usages
13
14
15  public class ProjektInfo {
16
17      @Id  2 usages
18      @GeneratedValue
19      private Long id;
20
21
22      private String projektName;     // اسم المشروع  3 usages
23      private String status;          // (on track, delayed :مثلاً) الحالة العامة  3 usages
24      private int fortschritt;          3 usages
25      private String startDatum;        3 usages
26      private String endDatum;          3 usages
27
28      @OneToMany(mappedBy = "projekt", cascade = CascadeType.ALL)  2 usages
29      private List<Task> tasks;   // Alle Tasks, die zu diesem Projekt gehören.
30
31
32
33      public ProjektInfo(String projektName, String status, int fortschritt, String startDatum, String endDatum) {  no usages
34
35          this.projektName = projektName;
36          this.status = status;
37          this.fortschritt = fortschritt;
38          this.startDatum = startDatum;
39          this.endDatum = endDatum;
40      }
```

Figure 6:Entity-Definition

10. A Projektstruktur

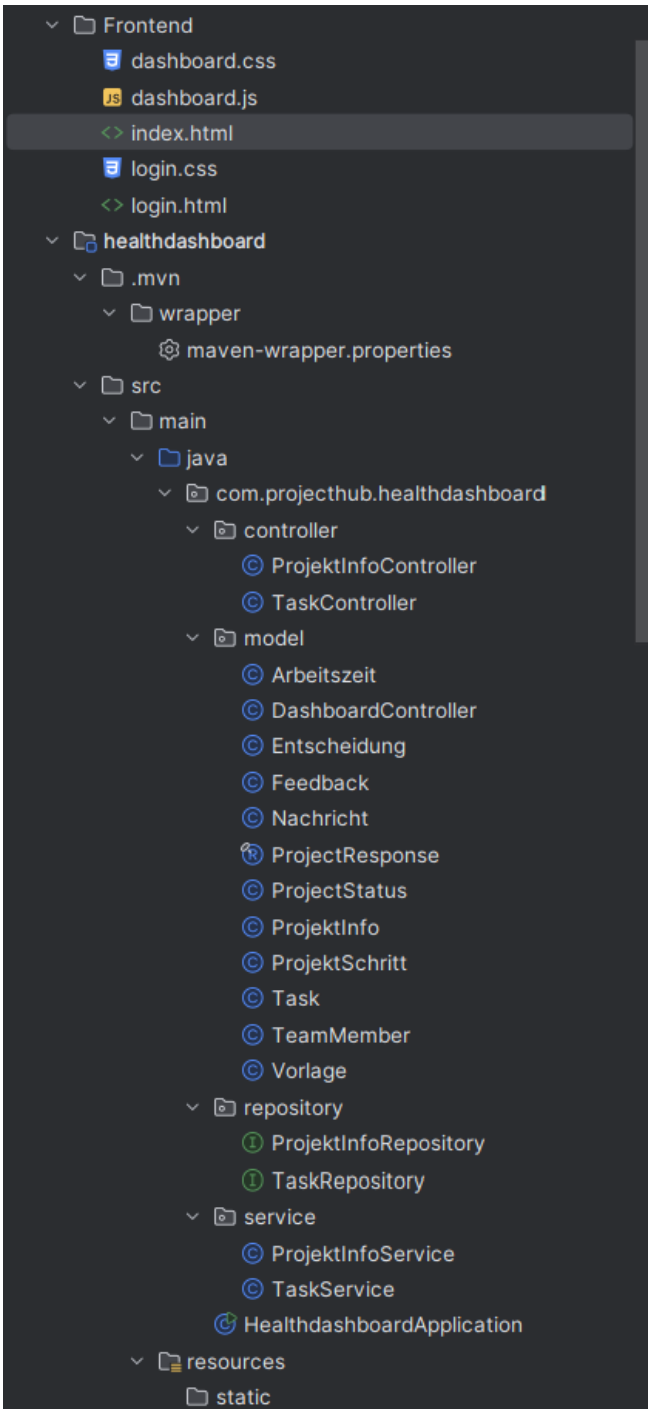

Figure 7: Projektstruktur

11. A Benutzeroberfläche

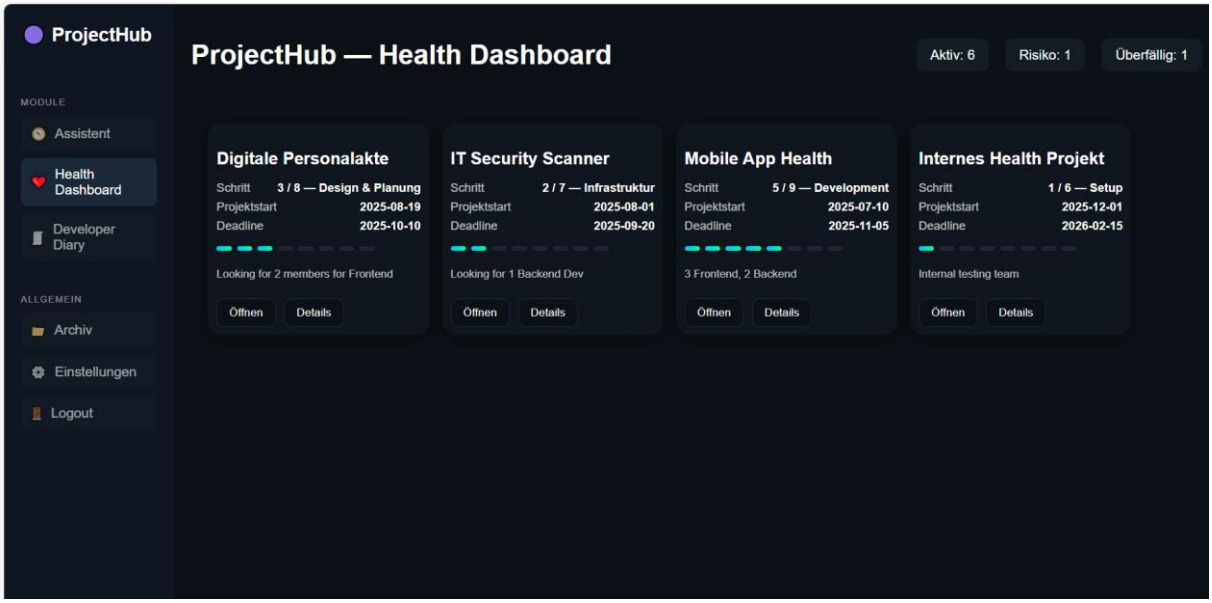

Figure 8:Benutzeroberfläche